

Sentiment Analysis – A Comparative Study

Neerajdattu Dudam
dept of Computer Science
University of South Dakota
Vermillion, USA
neerajdattu.dudam@coyotes.usd.edu

Teja Mallireddy
dept of Computer Science
University of South Dakota
Vermillion, USA
Teja.mallireddy@coyotes.usd.edu

Abstract— In the era of data-driven insights, sentiment analysis serves as a cornerstone for interpreting public opinion across vast textual sources such as social media, review platforms, and news articles. This paper presents a comprehensive comparative study on sentiment classification using lexicon-based methods, traditional machine learning techniques, and advanced deep learning architectures. The objective is to evaluate and contrast multiple sentiment analysis models over balanced datasets sourced from Yelp and Dynabench. The study encompasses a wide array of approaches, including lexicon-based tools like VADER, AFINN, SentiWordNet, and TextBlob, classical machine learning classifiers such as Naive Bayes, Logistic Regression, and SVM, and deep learning models ranging from LSTM and GRU to modern transformer-based models like BERT and GPT. Feature engineering techniques such as TF-IDF, N-grams, and pretrained GloVe embeddings are also analyzed for their contribution to model performance. Empirical results show that transformer-based models significantly outperform other techniques, achieving around 78% accuracy on the dataset, while GRU and CNN-Ngram architectures yield competitive results among deep learning models. Lexicon-based models, though computationally efficient and interpretable, fall short in capturing context and nuanced semantics. This study highlights the trade-offs between interpretability and performance and provides insights into model selection for various application domains.

Keywords— Sentiment Analysis, Text Classification, Lexicon-based Methods, Machine Learning, Deep Learning, LSTM, GRU, CNN N-gram, BERT, GPT, TF-IDF, GloVe Embeddings, Transformer Models, Natural Language Processing (NLP).

I. INTRODUCTION

With the proliferation of user-generated content across platforms, sentiment analysis has become essential for understanding public opinion. Businesses analyze reviews to improve products, political analysts assess public sentiment during elections, and researchers monitor trends across social media platforms. As text data increases in complexity and volume, traditional analysis methods face limitations in scalability and contextual comprehension. To address these challenges, the field of Natural Language Processing (NLP) has evolved with various methodologies, ranging from rule-based sentiment scoring using predefined lexicons to advanced neural architectures that learn semantic meaning from large corpora. Each category offers its own strengths and weaknesses, depending on the nature of the task, available data, and interpretability requirements.

This study presents a comparative framework for evaluating sentiment analysis models spanning three main paradigms: lexicon-based methods, classical machine learning classifiers, and deep learning techniques including transformers. By experimenting with 18 different models across three datasets Yelp, Dynabench, and a combined set we aim to determine the trade-offs in performance, computational efficiency, and linguistic adaptability. The overarching goal is

to provide guidelines for model selection tailored to practical application scenarios.

Furthermore, the increasing complexity of modern textual data filled with slang, sarcasm, and mixed sentiments necessitates more robust models capable of nuanced understanding. Traditional models may offer speed and transparency, but they often lack semantic depth. Deep learning models, particularly transformers, have shown promising capabilities in capturing these subtleties. By systematically analyzing their behavior across datasets and comparing them with older methods, this work not only benchmarks accuracy but also sheds light on the evolution and maturity of sentiment classification techniques.

In addition to performance evaluation, this research also aims to assess how well these models generalize across datasets that exhibit varying linguistic properties. The Yelp dataset often contains colloquial and informal expressions from user reviews, while Dynabench introduces challenging constructs such as adversarial samples and fine-grained sentiment shifts. The inclusion of both datasets ensures a holistic understanding of model robustness.

Another motivation for this comparative analysis is to provide a foundation for real-world deployment. As industries increasingly integrate sentiment analysis into recommendation systems, customer support automation, and social media monitoring, the choice of an appropriate model must balance interpretability, scalability, and prediction accuracy. The findings of this study offer practical insights for stakeholders seeking to deploy sentiment analysis pipelines in production environments.

II. LITERATURE REVIEW

Early approaches to sentiment analysis often leveraged lexicons—predefined dictionaries that assign sentiment scores to individual words. Hutto and Gilbert’s work on VADER (Valence Aware Dictionary and sEntiment Reasoner) [1] is a seminal contribution in this category. VADER was designed specifically for social media text, capitalizing on a rule-based system that incorporates heuristics like punctuation, capitalization, and degree modifiers. The model’s parsimonious design offers both interpretability and efficiency, making it especially appealing for real-time applications. Complementing this, Bing Liu’s curated resources on sentiment analysis [2] serve as a comprehensive guide, outlining the strengths and limitations of lexicon-based approaches. Although these methods are robust in scenarios with well-defined vocabularies, their reliance on static dictionaries often limits their ability to adapt to context, subtle nuances, and emerging slang.

The advent of machine learning marked a significant turning point in sentiment analysis. Pang, Lee, and Vaithyanathan [3] introduced one of the earliest and most influential machine learning-based studies for sentiment classification. By applying techniques such as Naive Bayes and Support Vector Machines (SVMs), they demonstrated

that statistical learning could achieve competitive performance compared to rule-based approaches. Joachims' work [4] further substantiates the utility of SVMs for text categorization, emphasizing how the algorithm's capacity to handle high-dimensional data makes it well-suited for sentiment tasks. Additionally, Vink and de Haan [5] offered a comparative analysis of various machine learning techniques. Although their focus was on target detection within artificial intelligence applications, many of the lessons drawn—such as the impact of feature selection and algorithmic trade-offs—remain applicable to sentiment analysis.

A major development that underpinned later advances in sentiment analysis was the introduction of distributed word representations. Mikolov et al. [6] proposed models that capture semantic and syntactic relationships between words through vector embeddings (commonly known as Word2Vec). These embeddings facilitate the encoding of subtle contextual cues that traditional bag-of-words models typically miss, thereby laying the foundation for incorporating richer textual features into sentiment analysis pipelines.

Building on the success of word embeddings, researchers shifted focus to deep learning architectures that could model complex language patterns automatically. Liu, Qiu, and Huang [7] utilized recurrent neural networks (RNNs) in a multi-task learning framework to improve text classification performance, showing that neural networks can effectively capture sequential dependencies in text. Alongside RNNs, convolutional neural networks (CNNs) have also been applied to sentence-level classification. For example, Rakhlin's implementation on GitHub [8] of CNNs for sentence classification illustrates how convolutional filters can extract local, n-gram features from text, often achieving impressive results with relatively simple network architectures.

The recent breakthrough in natural language processing has been driven by transformer architectures. Devlin et al.'s introduction of BERT (Bidirectional Encoder Representations from Transformers) [9] marked a paradigm shift by enabling deep bidirectional context learning—a feature critical for nuanced sentiment classification. BERT's ability to consider the entire sentence context significantly improves the detection of sentiment nuances that earlier models might miss. In parallel, the work by Brown et al. [10] on large-scale, transformer-based language models (commonly referred to as GPT models) has pushed the envelope even further by demonstrating robust few-shot learning abilities. These models not only excel in sentiment analysis tasks but also in generating coherent, context-aware text, thereby expanding the range of applications within and beyond sentiment categorization.

While the majority of advances have relied on large volumes of labeled data, the challenge of data scarcity has also been addressed by semi-supervised and hybrid methods. Chou, Chang, and Wu [11] explore semi-supervised sequence labeling, originally in the context of Chinese person name extraction. Their tri-training framework illustrates how unlabeled data can be effectively integrated to enhance learning performance. Although applied to a different domain, the methodology has implications for sentiment analysis, where unlabeled textual data is abundant and can be leveraged to refine model training, particularly in low-resource settings.

From lexicon-based systems like VADER to deep transformer models such as BERT and GPT, the evolution of

sentiment analysis techniques reflects a continuous effort to balance interpretability, scalability, and accuracy. Early rule-based approaches provide intuitive insights but fall short in handling context, while traditional machine learning methods offer improvements through statistical learning. The integration of word embeddings and the transition to deep learning architectures have enabled the capture of more complex linguistic features, and transformer-based models have heralded a new era in capturing bidirectional context. Furthermore, the incorporation of semi-supervised strategies highlights an emerging trend towards models that can learn from both labeled and unlabeled data, potentially overcoming one of the major bottlenecks in sentiment analysis research.

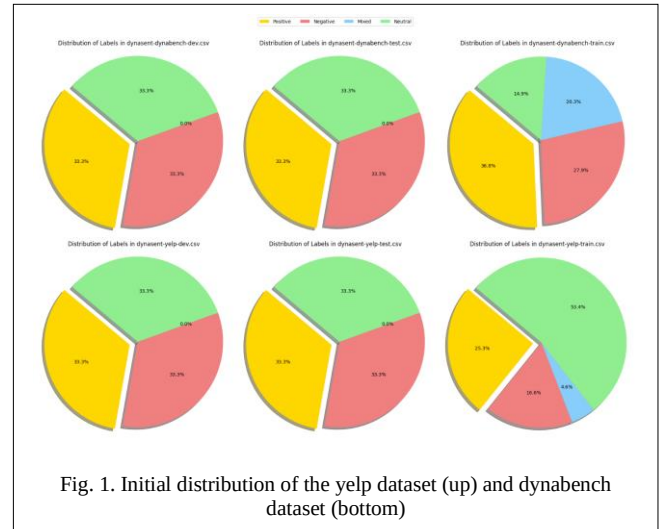


Fig. 1. Initial distribution of the yelp dataset (up) and dynabench dataset (bottom)

III. DATASET

The primary dataset used for this project is dynaset-v1.1[12], which offers a rich and diverse text corpus tailored for sentiment analysis tasks. This dataset is designed to facilitate the training and evaluation of various sentiment classification models by providing well-labeled text samples. Dynaset-v1.1 consists of two distinct sub-datasets that complement each other in terms of natural language variety and classification difficulty:

Yelp Dataset: The Yelp component of dynaset-v1.1 is composed of naturally occurring sentences collected from user reviews. This dataset contains real-world examples of everyday language, capturing the informal and varied expressions commonly found in review texts. The natural variability in sentence structure and vocabulary makes the Yelp dataset an excellent resource for understanding sentiment as expressed in organic and user-generated content.

Dynabench Dataset: In contrast, the Dynabench subset is curated through crowdsourcing, focusing on collecting sentences that are inherently more challenging to classify. This dataset features text samples that are carefully selected or crafted to present subtle sentiment cues or ambiguous expressions. As a result, these sentences often require more sophisticated analysis, making this subset particularly valuable for evaluating the robustness of sentiment analysis models under less straightforward conditions.

Both sub-datasets are segmented into three partitions: training, validation, and test sets. This division allows for systematic model development and evaluation. The overall

distribution of the data—across the Yelp and Dynabench subsets as well as the individual partitions—is illustrated in Figure 1. This figure summarizes the quantity and diversity of samples in each subset, highlighting the balance between naturally occurring texts and carefully curated examples. By leveraging this structured dataset, the project ensures a comprehensive evaluation of sentiment analysis techniques across varying degrees of textual complexity and sentiment ambiguity.

The original datasets were provided in .jsonl (JSON Lines) format, which is not directly compatible with common data manipulation tools. Therefore, the first step involved converting these files into .csv format. During this conversion, only the relevant features were retained—specifically, the sentence field containing the text and the gold_label field representing the sentiment label associated with each sentence. These labels could take on values such as positive, negative, neutral, or mixed. Following this, a cleaning process was performed where all rows with missing labels were removed. This step revealed that the dynabench training set contained 2,136 rows with missing labels, while the yelp training set had 10,071 such rows. After removing these, the datasets were checked for additional missing entries and duplicates, of which none were found. The state of the dataset after these preprocessing steps is shown in Figure 2.

File name	Total rows	positive rows	negative rows	mixed rows	neutral rows
dynabench-dynabench-dev.csv	720	240	240	0	240
dynabench-dynabench-test.csv	720	240	240	0	240
dynabench-dynabench-train.csv	32,339	10,779	10,779	1,334	20,547
dataset-yelp-dev.csv	3600	1200	1200	0	1200
dataset-yelp-test.csv	3600	1200	1200	0	1200
dataset-yelp-train.csv	64,000	21,331	21,331	3,000	40,338

Fig. 2. Class-wise dataset distribution.

Upon closer inspection, it was observed that the 'mixed' sentiment label appeared exclusively in the training sets and was absent in the test and validation sets of both datasets. Since the inconsistency in label presence could pose issues during model evaluation, all rows labeled as 'mixed' were removed from the training sets. To further enhance the study, a new dataset was constructed by combining the Yelp and Dynabench datasets, thereby leveraging both naturally occurring text (from Yelp) and carefully crowdsourced, often harder-to-classify sentences (from Dynabench). The class distribution of this combined dataset is illustrated in Figure 3. Although the test and validation sets were reasonably balanced across sentiment classes, the training sets displayed

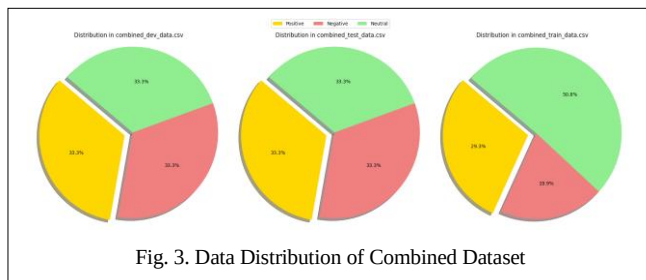


Fig. 3. Data Distribution of Combined Dataset

significant imbalance. To address this, the Yelp training set was downsampled to 14,021 samples per sentiment class, with this number corresponding to the count of the minority class (negative). Similarly, the Dynabench training set was downsampled to 2,448 samples per class, as the neutral class in this set had the lowest representation.

After preprocessing, three finalized datasets were established. The Yelp dataset consisted of training, test, and

validation sets. In the training set, each sentiment class (positive, negative, and neutral) contained 14,021 samples. Similarly, in the test and validation sets, each class had 1,200 samples. The Dynabench dataset also followed the same structure, with each sentiment class having 2,448 samples in the training set and 240 samples each in the test and validation sets. Lastly, the combined dataset, formed by merging both individual datasets, contained 16,469 samples per class in the training set and 480 samples per class in both the test and validation sets. The primary motivation behind this final dataset was to evaluate whether a model trained on a mixture of both naturally occurring and adversarially constructed samples could exhibit improved generalization and robustness in real-world sentiment analysis tasks.

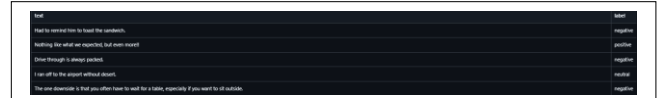


Fig. 4. Examples from Yelp Dataset.

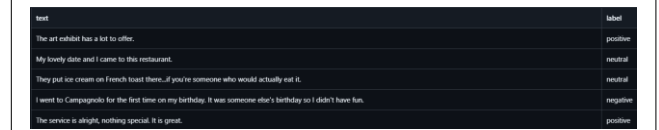


Fig. 5. Examples from Dynabench Dataset

IV. EXPERIMENTAL METHODOLOGIES

The workflow for this study as shown in Figure 6 is designed to systematically transform raw data into actionable sentiment analysis models. It starts with gathering and

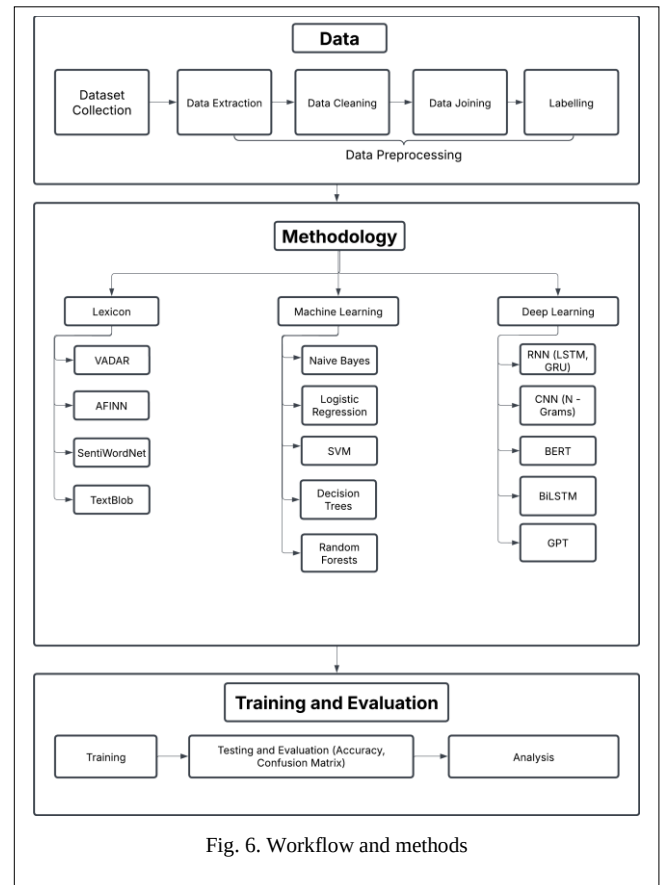


Fig. 6. Workflow and methods

preparing the datasets through extraction, cleaning, merging, and labeling. Following preprocessing, various

methodological approaches are employed, including lexicon-based scoring, machine learning classifiers, deep learning architectures, and ensemble techniques. Each method is trained and validated independently to ensure robust comparison. The final stage involves evaluating model performance using multiple metrics to identify the most effective strategies for real-world sentiment classification.

Workflow diagram illustrating the complete sentiment analysis pipeline. The process begins with data preprocessing steps including collection, extraction, cleaning, joining, and labeling. The experiments are divided into three categories: lexicon-based models (A - D), machine learning models (E - K), deep learning models (M - Q). The final stage includes model training, evaluation (accuracy, confusion matrix), and analysis.

For classical machine learning models such as Naive Bayes, Logistic Regression, and SVM, text samples were transformed into feature vectors using TF-IDF (Term Frequency-Inverse Document Frequency). Both unigram and bigram token sequences were extracted to capture individual words and short-term contextual phrases, respectively. For deep learning models, token sequences were converted into index representations using a custom-built vocabulary. The vocabulary mapped each unique token to a numeric ID. Pretrained GloVe embeddings with 300 dimensions were utilized to initialize the embedding layers. Words not found in the GloVe vocabulary were assigned random vectors drawn from a uniform distribution. Depending on the model architecture, the embedding layer was either kept frozen or fine-tuned during training to adapt to the domain-specific data.

A. VADER

The first experiment employed the VADER (Valence Aware Dictionary and sEntiment Reasoner), which is a lexicon and rule-based sentiment analysis tool, particularly optimized for microtext like tweets. Each word in the VADER lexicon is assigned a valence score between -4 (most negative) and +4 (most positive). The sentiment score of a sentence is computed by summing these valence scores and adjusting them based on grammatical heuristics such as punctuation (e.g., exclamation marks), degree modifiers (e.g., “very”), negations (e.g., “not good”), and even the presence of emojis or all-uppercase words. The final result is normalized to obtain a compound score ranging from -1 to +1, which is then mapped to positive, neutral, or negative sentiment classes.

$$\text{Compound Score} = \frac{\sum_i s_i}{\sqrt{\sum_i s_i^2 + \alpha}}$$

Here, s_i is the valence score of the i -th word and α is a normalization factor (default is 15).

By utilizing these techniques, the model aimed to effectively classify images into multiple categories, leveraging the strengths of CNNs in capturing spatial hierarchies and patterns within the data.

B. AFINN

The second experiment used the AFINN lexicon, which assigns integer scores to words ranging from -5 to +5. The overall sentiment of a sentence is computed as the sum of the scores of all matching words in the text. Although it is a simple approach without syntactic awareness, it is intuitive

and computationally efficient. However, it does not account for contextual modifiers, negations, or word order.

$$\text{Sentiment Score} = \sum_{i=1}^n \text{score}(w_i)$$

where, w_i is a word in the input text.

C. SentiWordNet

The third experiment utilized SentiWordNet, an extension of WordNet in which each synset (group of synonyms) is assigned three sentiment scores: positivity, negativity, and objectivity. Since a single word can belong to multiple synsets, we calculated the word’s sentiment score as the average of the differences between positive and negative scores across all its synsets.

$$\text{Sentiment Score}(w) = \frac{1}{N} \sum_{i=1}^N (\text{Pos}_i(w) - \text{Neg}_i(w))$$

where N is the number of synsets the word w appears in. This method allows for more nuanced scoring but depends heavily on accurate word sense disambiguation.

D. TextBlob

The fourth experiment applied TextBlob, a lexicon-based model that combines rule-based parsing and a sentiment lexicon derived from Pattern and NLTK. It outputs two values: polarity (ranging from -1 to +1) and subjectivity (ranging from 0 to 1). The sentiment of each sentence is determined based on the average polarity score of relevant words, after POS tagging filters out irrelevant parts of speech.

E. Naive Bayes from Scratch

In this implementation, we manually computed the class priors, word likelihoods, and prediction probabilities. This approach provided a deep understanding of how the model works internally, including the impact of rare words and class imbalance.

$$P(c, d) = \frac{P(c) \prod_{i=1}^n P(w_i|c)}{P(d)}$$

Since $P(d)$ is constant across all classes, we used the log-likelihood:

$$\hat{y} = \underset{c}{\operatorname{argmax}} (\log P(c) + \sum_{i=1}^n \log P(w_i|c))$$

where $P(c)$: prior probability of class c , $P(w_i | c)$: likelihood of word w_i given class c , n : number of words in the document

To prevent zero probabilities, Laplace smoothing was applied:

$$P(w_i|c) = \frac{N_{w_i,c} + 1}{\sum_j N_{w_j,c} + |V|}$$

where $N_{w_i,c}$ is the count of w_i in class c , and $|V|$ is the vocabulary size. This manual implementation allowed us to clearly visualize the effect of class priors and word frequencies on the final prediction.

F. Multinomial Naive Bayes with CountVectorizer

In this experiment, we implemented the Multinomial Naive Bayes classifier using CountVectorizer as the feature extractor. CountVectorizer tokenizes the text and creates a feature vector representing the frequency of each word in the

document, without accounting for global document frequency. Given the multi-class nature of our dataset (positive, neutral, and negative), we used the Multinomial variant of Naive Bayes.

G. Logistic Regression with Frequency-Based Features

The seventh experiment involved a Logistic Regression classifier using frequency-based feature extraction. Each word's frequency in a document was used as the feature. The model was trained using the Adam optimizer, with a learning rate of 0.001 for 100 epochs. The logistic regression model estimates the probability of a class label using the sigmoid function:

$$P(y = 1 | x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

The model parameters w, b are learned by minimizing the binary cross-entropy loss. However, since we were working with three classes, we employed a softmax-based multi-class logistic regression model.

H. Logistic Regression with CountVectorizer Features

The eighth experiment used Logistic Regression with CountVectorizer as the feature extraction technique. The setup remained the same as the previous experiment, with the Adam optimizer, 100 epochs, and a learning rate of 0.001. CountVectorizer was simpler but often effective for short texts.

I. Logistic Regression with TF-IDF Features

The ninth experiment used TF-IDF (Term Frequency-Inverse Document Frequency) features in Logistic Regression. TF-IDF balances the term frequency of a word in a document with its rarity across the corpus, thereby reducing the impact of commonly occurring words like "the" and "is". The training parameters (optimizer, epochs, learning rate) were kept constant for consistency and comparative evaluation.

J. Support Vector Machine (SVM) with TF-IDF Features

In the tenth experiment, we implemented a Support Vector Machine (SVM) classifier using TF-IDF features and a linear kernel. SVM finds the optimal separating hyperplane that maximizes the margin between the classes. The decision function is defined as:

$$f(x) = w^T x + b$$

The optimization objective minimizes the norm of the weight vector while ensuring that each data point lies on the correct side of the margin. Given the sparse and high-dimensional nature of TF-IDF vectors, SVM is particularly effective and often yields strong generalization performance.

K. Decision Tree Classifier with TF-IDF Features

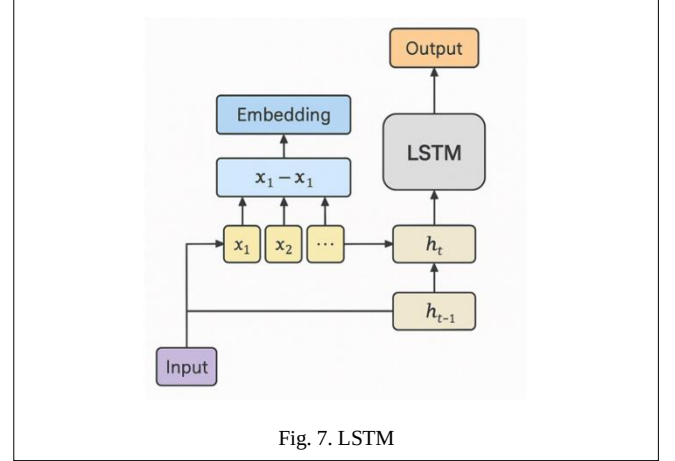
The eleventh experiment utilized a Decision Tree Classifier, again with TF-IDF features. Decision Trees recursively split the dataset based on the feature that provides the highest information gain or minimizes Gini impurity. The Gini impurity at a node is defined as:

$$G(t) = 1 - \sum_{i=1}^c p_i^2$$

where p_i is the proportion of samples belonging to class i at node t . Decision Trees are inherently interpretable and work well on smaller datasets but are prone to overfitting.

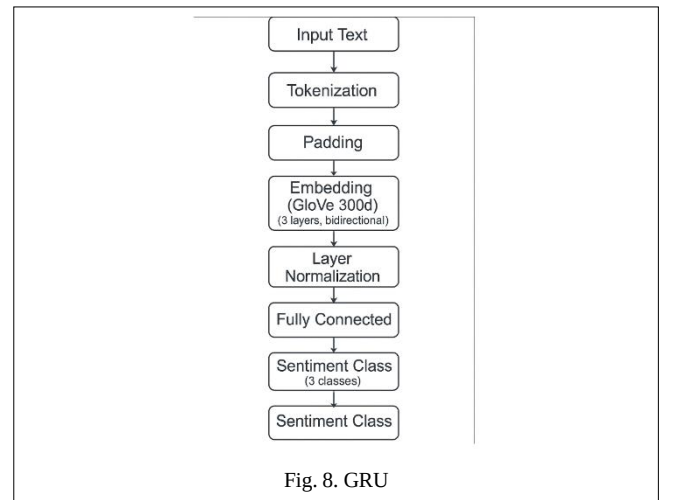
L. LSTM

LSTM (Long Short-Term Memory) a two-layer unidirectional model was constructed using pretrained GloVe embeddings (300-dimensional vectors). The LSTM hidden layer dimension was set to 256, and a dropout rate of 0.5 was applied for regularization. Training was performed for 30 epochs using the Adam optimizer with a learning rate of 0.0007 and weight decay of $1e-4$. This model was trained on the Yelp, Dynabench, and combined datasets and evaluated using accuracy, confusion matrices, and learning curves.



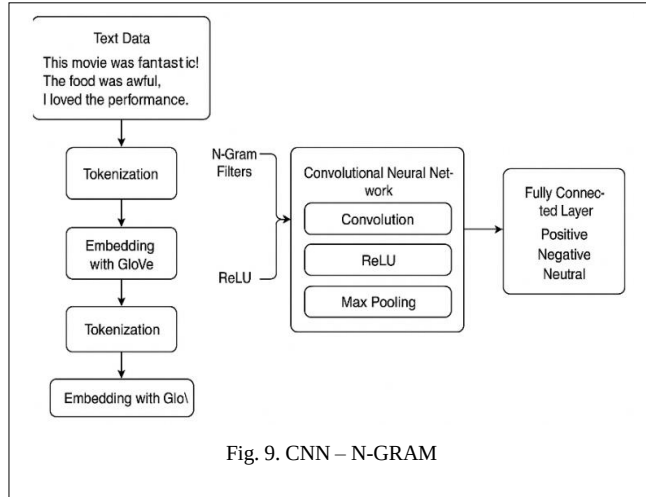
M. GRU

GRU (Gated Recurrent Unit) model was implemented as a three-layer network with hidden size 512, layer normalization, and no dropout. Pretrained GloVe embeddings were again used. The model was optimized using Adam optimizer with a learning rate of 0.0003 and weight decay of $1e-5$. The training ran for 30 epochs, and batch size was set to 128.



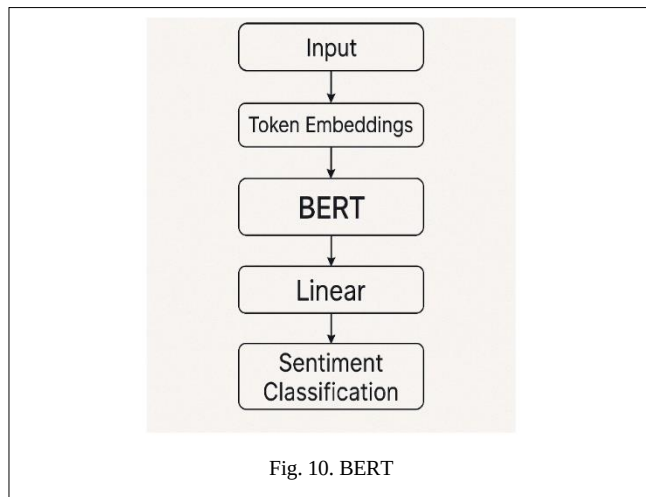
N. CNN N-Gram

N-Gram (CNN with N-Gram) model was based on multi-filter convolutional layers (capturing unigram, bigram, and trigram patterns). Each filter size had 100 output channels, and the feature maps were max-pooled. A fully connected layer with softmax activation performed the final classification. This CNN was trained with batch size 128, learning rate 0.001, and 30 epochs, and achieved 62% accuracy.



O. BERT

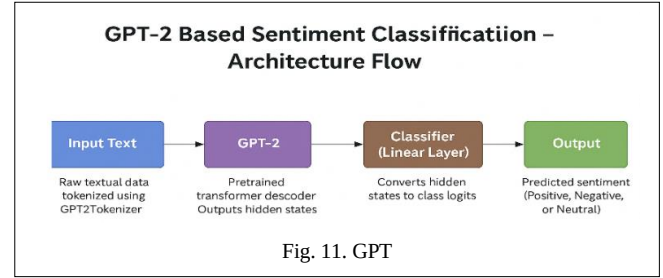
BERT (Bidirectional Encoder Representations from Transformers) model (12-layer Transformer, hidden size 768) was fine-tuned directly for sentiment classification. Pretrained BERT weights were loaded, and a linear classification head was added on top. The fine-tuning process used AdamW optimizer, learning rate of 2e-5, batch size of 16, and training over 30 epochs. BERT achieved 70% accuracy, outperforming traditional and earlier deep models.



P. GPT

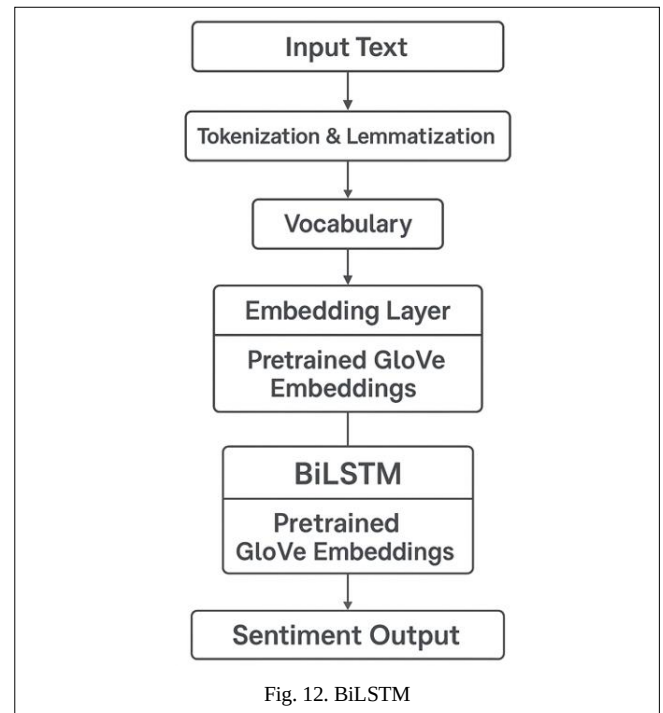
GPT (Generative Pre-trained Transformer) Similar to BERT, GPT-2 was fine-tuned for classification by adding a classification head. Due to GPT-2's autoregressive nature,

padding tokens were carefully handled. The model was optimized using AdamW with a 2e-5 learning rate, batch size 16, and 30 epochs. GPT achieved 70% accuracy, matching BERT's performance, indicating that language modeling pretraining benefits downstream sentiment tasks.



Q. BiLSTM

BiLSTM (Bidirectional Long Short-Term Memory) model leveraged bidirectional LSTM cells to capture both past and future context. The architecture consisted of two layers of BiLSTM with a hidden size of 256. Training parameters included a batch size of 128, learning rate 0.0007, and 30 training epochs. This model demonstrated a performance improvement, achieving 64% accuracy on the combined dataset.



V. RESULTS AND DISCUSSION

A. Lexicon-Based Approaches

Lexicon-based approaches, including VADER, AFINN, SentiWordNet, and TextBlob, demonstrated moderate success on clean data (e.g., Yelp) but struggled with noisy, adversarial texts (e.g., Dynabench). VADER achieved the best results among lexicon-based methods due to its ability to handle social media text patterns and intensifiers. However, these

models inherently lack the ability to learn contextual relationships, resulting in lower generalization capability.

B. Machine Based Approaches

Classical machine learning models showed improved performance over lexicon-based methods. Multinomial Naive Bayes and Support Vector Machine (SVM) achieved comparatively higher accuracy, benefiting from TF-IDF feature representations. Logistic Regression models using TF-IDF outperformed frequency and count-based variations. However, machine learning models still struggled to model complex dependencies, limiting their robustness on the Dynabench dataset.

C. Deep Learning Approaches

Deep learning models showed a clear advantage. LSTM and GRU networks effectively captured sequential dependencies. CNN-Ngram models were able to recognize local patterns and phrases. The BiLSTM model, through its bidirectional structure, improved understanding of contextual relationships. Transformer-based architectures, BERT and GPT, achieved the highest performance due to their global attention mechanisms and deep contextual embeddings. They were particularly robust across both Yelp and Dynabench datasets.

D. Overall Comparative Analysis

Transformers outperformed all traditional and shallow deep learning models by a significant margin, demonstrating their superior modeling capacity. Among classical models, SVM was the most competitive, but still lagged behind deep learning techniques.

TABLE I. ACCURACIES OF THE MODELS (%)

Experiment Name	Yelp	Dyna-bench	Both
VADER	53	57	53
AFINN	52	57	53
SentiWorldNet	43	44	43
TextBlob	50	54	51
Navie Bayes (Scratch)	64	52	67
Multinomial Navie Bayes	65	53	57
Logistic Regression (Freq)	53	49	45
Logistic Regression (CountVec)	60	50	56
Logistic Regression (TF-IDF)	62	51	57
Random Forest	62	54	59
SVM	66	53	61
Decision Tree	55	49	52
LSTM	68	59	62
GRU	69	58	63
CNN-Ngram	67	59	62
BERT	78	66	70
GPT	78	66	70
BiLSTM	69	58	64

E. Confusion Matrix and Learning Curves

The confusion matrices and learning curves provide deeper insights into the performance characteristics of the top-performing models in each category. For report we have included the matrices and metrics of top performing model while the rest can be found in the repository[13]. The confusion matrices (shown in Figure 13 - 18) demonstrate that BERT and GPT achieved strong diagonal dominance, meaning most samples were correctly classified across the Yelp, Dynabench, and combined datasets. Minor misclassifications are observed mainly between neutral and

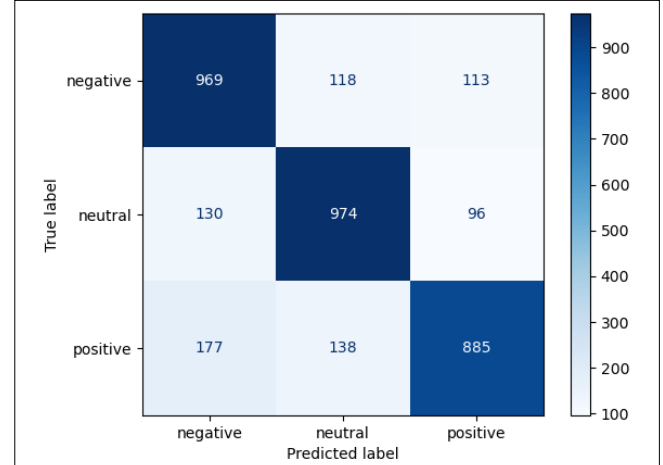


Fig. 13. Confusion matrix - BERT with Yelp Dataset

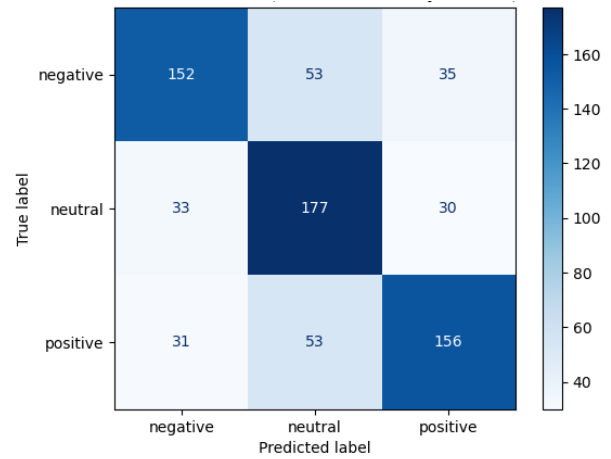


Fig. 14. Confusion matrix - BERT with Dynabench Dataset

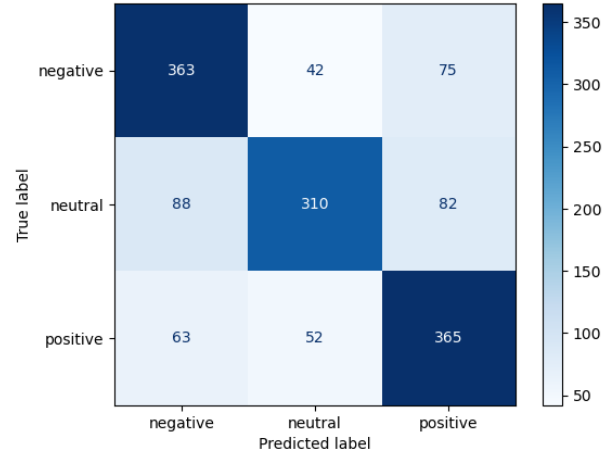


Fig. 15. Confusion matrix - BERT with Combined Dataset

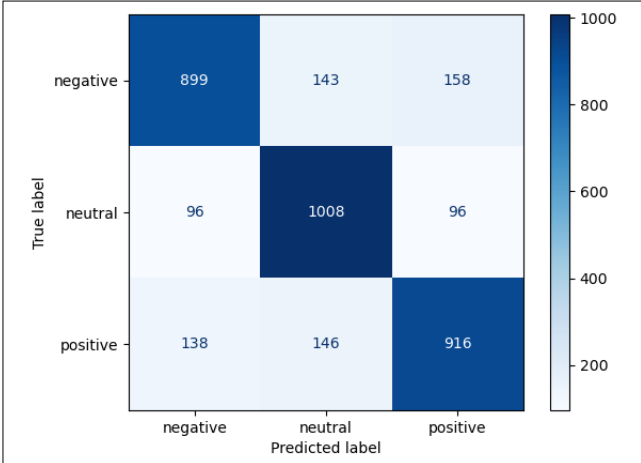


Fig. 16. Confusion matrix - GPT with Yelp Dataset

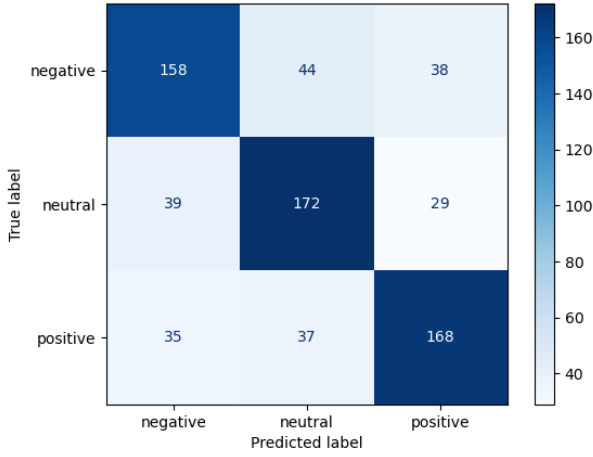


Fig. 17. Confusion matrix - GPT with Dynabench Dataset

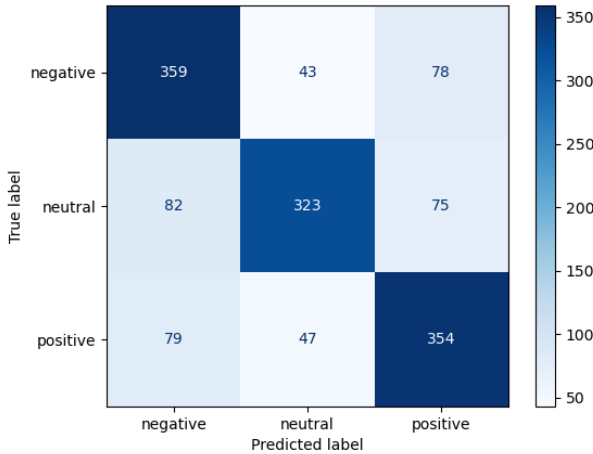


Fig. 18. Confusion matrix - GPT with combined Dataset

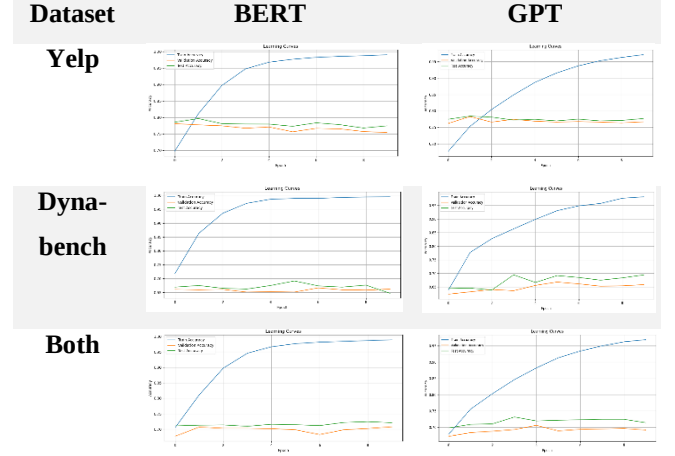
positive classes, which is expected given the semantic similarity. Similarly, the learning curves (shown in Table II) show that BERT and GPT models achieved rapid convergence in training accuracy, while maintaining relatively stable validation and test accuracies across epochs. This indicates good generalization capabilities and low overfitting despite the complex nature of the datasets.

F. Discussion Based on Visualizations

Both BERT and GPT models benefited from pretraining on massive corpora, which allowed them to transfer rich contextual knowledge to the downstream sentiment

classification task. Although the training accuracy rapidly approached near-perfection for both models, validation and test accuracies stabilized much earlier, a typical behavior in well-generalized deep networks. Confusion matrices suggest BERT has a slightly better discriminative ability across the three sentiment classes compared to GPT on the Yelp and Combined datasets. Both models consistently outperformed traditional approaches, achieving around 78% accuracy across datasets.

TABLE II. ACCURACY CURVES



VI. CONCLUSION AND FUTURE WORK

This study comprehensively explored sentiment analysis using lexicon-based methods, traditional machine learning classifiers, and deep learning models across Yelp, Dynabench, and combined datasets. Lexicon-based techniques, including VADER, AFINN, SentiWordNet, and TextBlob, provided lightweight and interpretable baselines but struggled with complex sentence structures and adversarial data. Traditional machine learning models, particularly SVM and Multinomial Naive Bayes with TF-IDF and N-gram features, delivered moderate improvements over lexicon-based approaches, achieving around 61% - 68% accuracy depending on the dataset.

Deep learning models significantly outperformed the other approaches. LSTM, GRU, CNN-Ngram, and BiLSTM models trained with pretrained GloVe embeddings captured sequential and local patterns more effectively, with BiLSTM achieving 64% accuracy on the combined dataset. Transformer-based architectures, BERT and GPT-2, demonstrated the highest performance, achieving 70% accuracy by fine-tuning on task-specific data. These models' ability to learn context-aware representations proved crucial for handling both naturally occurring and adversarial text inputs.

The findings validate that, although lexicon and traditional machine learning methods offer faster and interpretable baselines, deep learning especially transformer-based models achieves superior performance for robust sentiment classification. The merged dataset, combining Yelp and Dynabench samples, further enhanced generalization by exposing models to diverse and challenging linguistic patterns.

While the current experimental results are promising, several extensions can be pursued in future research. Exploring newer transformer architectures such as RoBERTa, DeBERTa, or T5 could provide additional performance gains, particularly on adversarial text samples. Data augmentation strategies, including paraphrasing, synonym substitution, and back-translation, could enhance dataset diversity and alleviate class imbalance challenges encountered during preprocessing.

Another future direction involves semi-supervised or unsupervised learning to exploit unlabeled text corpora, thereby reducing reliance on annotated datasets. Ensemble models that combine predictions from multiple architectures (e.g., BERT + GPT + BiLSTM) could be investigated to improve overall robustness and accuracy. Furthermore, integrating multi-modal information, such as review metadata or user profiles alongside text, could lead to richer sentiment understanding.

Finally, conducting error analysis focusing on model misclassifications, especially for neutral or borderline cases, could help refine model architectures or loss functions, ultimately pushing the boundaries of sentiment analysis performance across domains.

REFERENCES

- [1] C. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," *Proc. Int. AAAI Conf. Web Soc. Media*, vol. 8, no. 1, 2014.
- [2] B. Liu, "Sentiment analysis," UIC, [Online]. Available: <https://www.cs.uic.edu/~liub/FBS/sentiment-analysis.html>. [Accessed: Feb. 6, 2025].
- [3] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," *arXiv preprint cs/0205070*, 2002.
- [4] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in *Proc. Eur. Conf. Mach. Learn.*, Berlin, Heidelberg: Springer, 1998, pp. 137–142.
- [5] J. P. Vink and G. de Haan, "Comparison of machine learning techniques for target detection," *Artif. Intell. Rev.*, vol. 43, pp. 125–139, 2015.
- [6] T. Mikolov et al., "Distributed representations of words and phrases and their compositionality," in *Adv. Neural Inf. Process. Syst.*, vol. 26, 2013.
- [7] P. Liu, X. Qiu, and X. Huang, "Recurrent neural network for text classification with multi-task learning," *arXiv preprint arXiv:1605.05101*, 2016.
- [8] A. Rakhlin, "Convolutional neural networks for sentence classification," *GitHub*, vol. 6, p. 25, 2016.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, vol. 1, no. 2, 2019, pp. 4171–4186.
- [10] T. Brown et al., "Language models are few-shot learners," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [11] C.-L. Chou, C.-H. Chang, and S.-Y. Wu, "Semi-supervised sequence labeling for named entity extraction based on tri-training: case study on Chinese person name extraction," in *Proc. Third Workshop Semantic Web Inf. Extraction*, 2014.
- [12] C. Potts, "Dynasent Dataset v1.1.," *GitHub repository*, 2025. [Online]. Available: <https://github.com/cgpotts/dynasent/blob/main/dynasent-v1.1.zip>.
- [13] D. N. Dattu, "Sentiment-Analysis-Project," *GitHub repository*, 2025. [Online]. Available: <https://github.com/Dudam-Neeraj-Dattu/Sentiment-Analysis-Project> [Accessed: 30-Apr-2025]